T02 Directional Animation

CRE132 worksheet

What you'll make

T01's player, now with the right animation for each direction: walking up shows the back, walking down shows the front, and left and right show the side. When you stop, the player keeps facing the way it last walked.

You'll make eight animation clips from the sprite sheet, put them into two **2D blend trees**, and write a short script that drives them. By the end you'll know how a 2D blend tree picks a clip from a direction, and how one small variable lets a character remember which way it was facing.



Figure 1: The finished meadow: the player facing down on the path, with trees, bushes and a rock around it.

Before you start

1. Unzip `CRE132_T02_DirectionalAnimation_Starter.zip`. In Unity Hub, choose **Add > Add project from disk** and pick the `CRE132_T02_DirectionalAnimation_Starter` folder. Open it with Unity **6000.3.11f1**.
2. Open the scene `Assets/_game/Scenes/TopDown/T02_DirectionalAnimation`.
3. Press **Play**. You can walk in eight directions (WASD, arrows, stick or D-pad) with T01's **TopDownMovement**, but the player is a still picture facing down: nothing animates. Press Play again to stop.

The **Player** has an **Animator** component, but its **Controller** slot is empty, and there are no clips for it yet. The sprite sheet is `Assets/_game/Art/Sprites/MysticWoods/player.png`, already cut into frames numbered `player_0`, `player_1` and so on, left to right and row by row. You'll make the clips and the controller, then write the script that drives them.

Making a script: in the Project window, go to the folder, right-click and choose **Create > Scripting > MonoBehaviour Script**. Type the name straight away and press Enter. The name must match the class name in the code exactly, capital letters included.

1. Eight clips

You made clips by hand in C07. This time there are eight: **Idle** and **Walk**, each facing **Down**, **Right**, **Left** and **Up**. First make the controller they'll belong to.

1. In the Project window, go to `Assets/_game/Animation/Controllers`. Right-click and choose **Create > Animation > Animator Controller**. Name it `PlayerDirectional`.
2. Select the **Player** in the Hierarchy and drag `PlayerDirectional` into the **Controller** slot of its **Animator** component.
3. Open **Window > Animation > Animation** and dock it at the bottom. With the Player selected, it says there's no clip yet and shows a **Create** button. Click **Create**, go to `Assets/_game/Animation/Clips/MysticWoods` and save the clip as `Player_Idle_Down`.
4. Set **Samples** to **8**. (If you can't see it, open the window's menu, the three dots at the top right, and choose **Show Sample Rate**.)
5. In the Project window, open `Assets/_game/Art/Sprites/MysticWoods` and click the arrow on `player.png` to show its frames. Click `player_0`, Shift-click `player_5`, and drag all six onto the timeline in the Animation window. A **Sprite** track appears with a diamond (a key) for each frame.
6. Click **Add Property**, open **Sprite Renderer** and click the **+** next to **Flip X**. A **Flip X** track appears with a key at each end, its box unticked: this clip keeps Flip X off.

Make the other seven the same way. Open the clip drop-down at the top left of the Animation window (it shows **Player_Idle_Down**), choose **Create New Clip...**, save it in the same `MysticWoods` folder with the name below, then do steps 4 to 6 with its samples and frames:

Clip	Frames	Samples	Flip X
`Player_Idle_Down`	`player_0` to `player_5`	8	off
`Player_Idle_Right`	`player_6` to `player_11`	8	off
`Player_Idle_Left`	`player_6` to `player_11`	8	**on**
`Player_Idle_Up`	`player_12` to `player_17`	8	off
`Player_Walk_Down`	`player_18` to `player_23`	10	off
`Player_Walk_Right`	`player_24` to `player_29`	10	off
`Player_Walk_Left`	`player_24` to `player_29`	10	**on**
`Player_Walk_Up`	`player_30` to `player_35`	10	off

For the two Left clips, switch Flip X on after adding the track: with the white playhead line on the first frame, tick the box in the **Flip X** row. Then drag the playhead to the last Flip X key and tick the box again. Both keys are now on, and the preview in the Scene view faces left.

- **The sprite sheet only has right-facing frames.** The Left clips use the right frames and switch **Flip X** on, which saves drawing a whole left-facing set.
- **Every other clip keys Flip X off.** Otherwise, after walking left and then up, the sprite would stay flipped.

Test it: press Play. The player now breathes facing down, whichever way you walk. Each clip you created also became a state in `PlayerDirectional`, and the first one, **Player_Idle_Down**, plays because it's the default. Press Play again to stop.

2. Two blend trees

Open the **Animator** window (**Window > Animation > Animator**) with the Player selected. It shows eight states, one per clip. You'll replace them with two states that each choose a clip by direction.

1. Drag a box round the eight states to select them all and press **Delete**. Only the states go: the clips stay in the Project window.
2. On the **Parameters** tab at the left, click **+**, choose **Float** and name it `Speed`. Add two more Floats the same way: `DirectionX` and `DirectionY`.
3. Right-click the empty grid and choose **Create State > From New Blend Tree**. Select the new state and rename it `Idle` at the top of the Inspector. It's orange: the default state. (If it isn't, right-click it and choose **Set as Layer Default State**.)
4. Double-click **Idle** to open it, and click the **Blend Tree** node. In the Inspector, set **Blend Type** to **2D Simple Directional**. Two parameter drop-downs appear under it: set the first to **DirectionX** and the second to **DirectionY**.
5. Under **Motion**, click **+ > Add Motion Field** four times. Drag a clip from `Clips/MysticWoods` into each row and type its **Pos X** and **Pos Y**:

Motion	Pos X	Pos Y
`Player_Idle_Down`	0	−1
`Player_Idle_Right`	1	0
`Player_Idle_Left`	−1	0
`Player_Idle_Up`	0	1

The diagram above the list shows each clip as a blue dot, and the current direction as a red dot. Drag the red dot around the diagram and watch the preview at the bottom of the Inspector turn the player.

6. Click **Base Layer** at the top of the Animator window to go back. Make a second blend tree the same way (steps 3 to 5), name it `Walk`, and give it the four **Player_Walk_** clips at the same positions: Down (0, −1), Right (1, 0), Left (-1, 0) and Up (0, 1).

C12 used a 1D blend tree, which picks a clip from one number. A 2D tree picks a clip from a direction: send it (0, 1) and it plays the Up clip.

Transitions: go back to the **Base Layer**.

1. Right-click **Idle**, choose **Make Transition** and click **Walk**. Select the arrow. In the Inspector, untick **Has Exit Time**, open **Settings** and set **Transition Duration** to **0**. Under **Conditions**, click **+** and set it to **Speed**, **Greater**, **0.1**.
2. Make a transition from **Walk** back to **Idle** the same way, with the condition **Speed**, **Less**, **0.1**.

- **Has Exit Time** off and **Transition Duration** 0 mean the animation changes the moment the player starts or stops.

Test it: press Play. The player breathes facing down but never walks or turns: nothing sets the three parameters yet. That's the script's job.

3. The DirectionalAnimator script

Make a folder `Assets/_game/Scripts/T02_DirectionalAnimation` and create `DirectionalAnimator` in it:

```
// Sends the direction of movement to the Animator's blend trees (DirectionX, DirectionY)
// and the speed (Speed). It remembers the last direction, so the character keeps facing
// that way when it stops. Put it on the object with the Animator; it finds the
```

```
// Rigidbody2D on itself or a parent.
using UnityEngine;

public class DirectionalAnimator : MonoBehaviour
{
    private Rigidbody2D body;
    private Animator animator;

    // Facing down to start with: (0, -1).
    private Vector2 lastDirection = Vector2.down;

    // Other scripts (the T04 attack) use this to know which way the character faces.
    public Vector2 LastDirection
    {
        get { return lastDirection; }
    }

    private void Awake()
    {
        body = GetComponentInParent<Rigidbody2D>();
        animator = GetComponent<Animator>();
    }

    private void Update()
    {
        Vector2 velocity = body.linearVelocity;
        float speed = velocity.magnitude;

        if (speed > 0.1f)
        {
            // Pixel art can't blend between two directions, so pick the main one:
            // left/right if moving more sideways, up/down if moving more vertically.
            if (Mathf.Abs(velocity.x) > Mathf.Abs(velocity.y))
            {
                lastDirection = new Vector2(Mathf.Sign(velocity.x), 0f);
            }
            else
            {
                lastDirection = new Vector2(0f, Mathf.Sign(velocity.y));
            }
        }

        animator.SetFloat("Speed", speed);
        animator.SetFloat("DirectionX", lastDirection.x);
        animator.SetFloat("DirectionY", lastDirection.y);
    }
}
```

What's going on:

- **It reads the Rigidbody2D's velocity**, not the keys, so it works with any movement script, and later with enemies too.
- **`lastDirection` only changes while moving.** When you stop, `speed` drops below 0.1, the `if` is skipped, and the old direction is sent again. That's all "remember which way I was facing" takes.
- **It snaps to four directions.** Pixel-art clips can't be mixed, so walking diagonally would sit halfway between two clips and flicker. `Mathf.Abs` compares how much of the movement is sideways and how much is up or down, and the bigger one wins. `Mathf.Sign` turns any number into 1 or −1.

- **`LastDirection` is a read-only property**, like `CurrentHealth` in S01. T04's attack uses it to hit the way the player faces.
- **The names in `SetFloat` must match the Animator's parameters exactly**, capital letters included.

Add it: select the **Player** and click **Add Component > Directional Animator**. There's nothing to set: it finds the Animator and the Rigidbody 2D itself.

Test it: press Play and walk in each direction, then stop. The player keeps facing the way it last walked.

4. Watching the blend tree

Press Play, select the **Player** and open the **Animator** window next to the Game view. Double-click **Walk** and walk around: a red dot moves between the four clips as **DirectionX** and **DirectionY** change. Walk diagonally and the dot stays on one of the four, because the script snaps the direction.

Check your work

- ☐ `PlayerDirectional` has two states, **Idle** and **Walk**, each a 2D blend tree with four clips, and three Float parameters.
- ☐ Walking up, down, left and right plays the matching walk clip.
- ☐ Stopping plays the idle clip facing the same way.
- ☐ Walking left then up doesn't leave the sprite flipped.
- ☐ Walking diagonally shows one direction steadily, with no flicker.
- ☐ Compare with `CRE132_T02_DirectionalAnimation_Finished` if something behaves differently.

Key points

- **A 2D blend tree picks a clip by a direction** (two numbers) instead of a single number like C12's 1D tree.
- **"Remember the last direction" is just a variable** that isn't updated when you stop.
- **Flip X in the clips** saves drawing a whole set of left-facing frames.

If something goes wrong

- **The player stays a still picture:** the **Controller** slot on the Player's **Animator** is empty. Drag `PlayerDirectional` into it.
- **The player always faces down, or never walks:** a name in `SetFloat` doesn't match the Animator's parameter. "Directionx" and "DirectionX" are different.
- **Walking one way shows another way's clip:** a clip has the wrong **Pos X** or **Pos Y** in its blend tree. Check both trees against the table in part 2.
- **The Left clips face right:** their **Flip X** keys aren't ticked. Pick each one in the Animation window and tick both keys.
- **The sprite stays flipped after walking left:** one of the other clips is missing its **Flip X** track. Add it with **Add Property** and leave it off.
- **A clip is far too fast:** its **Samples** is still 60. Set it to 8 (Idle) or 10 (Walk).
- **There's an extra Blend parameter:** Unity added it for a blend tree. Delete it and set the tree's parameters to **DirectionX** and **DirectionY**.
- **NullReferenceException in DirectionalAnimator:** the script isn't on the object with the **Animator** and **Rigidbody 2D**.
- **The animation changes late or slides into the next one:** select the transition arrow and switch off **Has Exit Time**, and set **Transition Duration** to 0.

Try it

1. Make diagonals prefer left and right instead of up and down. (Hint: one character in the `if`.)
2. Add a run animation: a third blend tree with the walk clips at a higher **Speed**, used when the player moves fast.
3. Use the stick direction instead of the velocity, so the player can turn on the spot without moving.
4. Change the four Walk clips' **Samples** to 14. Does the walk look better or worse? Why might a game use a different rate for walking and running?